

Note 3.1 Algorithms

Introduction

In this section, we will learn about the corresponding algorithm in discrete mathematics.

Definition of Algorithm

An **algorithm** is a **finite** sequence of precise instructions for performing a computation or for solving a problem.

Definition of Pseudocode

The code in English language description rather than a particular computer language.

Algorithm 1--Finding the Maximum Element in a Finite Sequence

1. Set the temporary maximum equals to the first integer.
2. Compare the next integer to the temporary maximum. If it is larger than the temporary maximum, renew the maximum.
3. Repeated the previous step until no integers are left in the sequence.

In pseudocode:

ALGORITHM 1 Finding the Maximum Element in a Finite Sequence.

```
procedure  $\max(a_1, a_2, \dots, a_n$ : integers)
 $\max := a_1$ 
for  $i := 2$  to  $n$ 
    if  $\max < a_i$  then  $\max := a_i$ 
return  $\max$ { $\max$  is the largest element}
```

Properties of Algorithms

- **input**. It must has input values.
- **output**. The output are the solution to the problem.
- Definiteness. The steps should be defined precisely.
- Correctness.
- **Finiteness**. It must contains finite steps.
- Effectiveness.
- Generality.

Searching Algorithm

Searching Problems

The problem of **locating** an element in an **ordered** list.

The Linear Search (Sequential Search)

Compare the key value x and the first element in the sequence. If they are equivalent then the solution is the location of the element. If not continue to the next element until found. If without locating x , the solution is the message that not found.

In Pseudocode:

ALGORITHM 2 The Linear Search Algorithm.

```
procedure linear search( $x$ : integer,  $a_1, a_2, \dots, a_n$ : distinct integers)
 $i := 1$ 
while ( $i \leq n$  and  $x \neq a_i$ )
     $i := i + 1$ 
if  $i \leq n$  then  $location := i$ 
else  $location := 0$ 
return  $location$  { $location$  is the subscript of the term that equals  $x$ , or is 0 if  $x$  is not found}
```

The Binary Search

It proceeds by comparing the element to be located to the **middle term** of the list and then **split into two smaller sublists of the same size** until find the key value.

Note that the sequence need to be **descending** or **ascending**.

In Pseudocode:

ALGORITHM 3 The Binary Search Algorithm.

```
procedure binary search ( $x$ : integer,  $a_1, a_2, \dots, a_n$ : increasing integers)
 $i := 1$  { $i$  is left endpoint of search interval}
 $j := n$  { $j$  is right endpoint of search interval}
while  $i < j$ 
     $m := \lfloor (i + j) / 2 \rfloor$ 
    if  $x > a_m$  then  $i := m + 1$ 
    else  $j := m$ 
if  $x = a_i$  then  $location := i$ 
else  $location := 0$ 
return  $location$  { $location$  is the subscript  $i$  of the term  $a_i$  equal to  $x$ , or 0 if  $x$  is not found}
```

Sorting

Definition of Sorting

Sorting is putting the elements in the list into the list whose elements are ordered (generally ascending).

Bubble Sort

It is one of the simplest sorting algorithms but not one of most efficient.

It interchanges a larger element with a smaller one following it, starting at the beginning for a pull pass. And iterate the procedure until the sort is complete.

In Pseudocode

ALGORITHM 4 The Bubble Sort.

```
procedure bubblesort( $a_1, \dots, a_n$  : real numbers with  $n \geq 2$ )  
for  $i := 1$  to  $n - 1$   
    for  $j := 1$  to  $n - i$   
        if  $a_j > a_{j+1}$  then interchange  $a_j$  and  $a_{j+1}$   
{ $a_1, \dots, a_n$  is in increasing order}
```

Insertion Sort

Simple but not efficient.

It begins with the second element and then compare with the first element. If it is less than the first element we insert it before the first element. If not we insert it after the first element. Thus the first two elements are ordered. Then we move the lower bound to the next and then compare with the former elements and insert it into the correct position.

In Pseudocode

ALGORITHM 5 The Insertion Sort.

```
procedure insertion sort( $a_1, a_2, \dots, a_n$  : real numbers with  $n \geq 2$ )  
for  $j := 2$  to  $n$   
     $i := 1$   
    while  $a_j > a_i$   
         $i := i + 1$   
     $m := a_j$   
    for  $k := 0$  to  $j - i - 1$   
         $a_{j-k} := a_{j-k-1}$   
     $a_i := m$   
{ $a_1, \dots, a_n$  is in increasing order}
```

Greedy Algorithms

Optimization Problems

The goal of such problem is to find a solution to given problem that either minimizes or maximizes the value of some parameters.

Greedy Algorithms

Select the best choice at each step instead of considering all sequences of steps that may lead to an optimal solution. And if we use greedy algorithm to find a feasible solution, we need to check whether it is an optimal solution.

Example of Greedy Change-Making Algorithm

EXAMPLE 6 Consider the problem of making n cents change with quarters, dimes, nickels, and pennies, and using the least total number of coins. We can devise a greedy algorithm for making change for n cents by making a locally optimal choice at each step; that is, at each step we choose the coin of the largest denomination possible to add to the pile of change without exceeding n cents. For example, to make change for 67 cents, we first select a quarter (leaving 42 cents). We next select a second quarter (leaving 17 cents), followed by a dime (leaving 7 cents), followed by a nickel (leaving 2 cents), followed by a penny (leaving 1 cent), followed by a penny. 

The greedy algorithm here is that we can add the amount of the largest number until it will exceed n , then we add the amount of the second largest number until it will exceed the rest and continue the procedure until their sum equals n .

In Pseudocode:

ALGORITHM 6 Greedy Change-Making Algorithm.

procedure *change*($c_1, c_2, \dots, c_r$: values of denominations of coins, where

$c_1 > c_2 > \dots > c_r$; n : a positive integer)

for $i := 1$ **to** r

$d_i := 0$ { d_i counts the coins of denomination c_i used}

while $n \geq c_i$

$d_i := d_i + 1$ {add a coin of denomination c_i }

$n := n - c_i$

{ d_i is the number of coins of denomination c_i in the change for $i = 1, 2, \dots, r$ }

The greedy algorithm can be proved as the optimal algorithm on page 199, 200. Here we omit the proof.

Example of Scheduling Talks

EXAMPLE 7 Suppose we have a group of proposed talks with preset start and end times. Devise a greedy algorithm to schedule as many of these talks as possible in a lecture hall, under the assumptions that once a talk starts, it continues until it ends, no two talks can proceed at the same time, and a talk can begin at the same time another one ends. Assume that talk j begins at time s_j (where s stands for *start*) and ends at time e_j (where e stands for *end*).

Here we have three choices to determine the greedy algorithm: the earliest start time, the shortest time, the **earliest end time**. Here the first two choice can take counterexamples and the third is right and the proof is in chapter 5.

In Pseudocode

ALGORITHM 7 Greedy Algorithm for Scheduling Talks.

```
procedure schedule( $s_1 \leq s_2 \leq \dots \leq s_n$ : start times of talks,  
     $e_1 \leq e_2 \leq \dots \leq e_n$ : ending times of talks)  
    sort talks by finish time and reorder so that  $e_1 \leq e_2 \leq \dots \leq e_n$   
     $S := \emptyset$   
    for  $j := 1$  to  $n$   
        if talk  $j$  is compatible with  $S$  then  
             $S := S \cup \{\text{talk } j\}$   
    return  $S$  { $S$  is the set of talks scheduled}
```

The Halting Problem

It input a computer program and the input of the computer program to the procedure and determines whether the program will eventually stop when run with this input.

It has been proved there are **no algorithms** to solve this problem. So we will know that there exist problems that cannot be expressed by any algorithm.